

A Physics-Informed Neural Network for Option Pricing under Rough Volatility: Meshfree Solution of the Markovian-Lifted PDE and Fast Parametric Calibration

[Authors / affiliations]

Complete draft (IMRAD). July 6, 2026

Abstract

Rough-volatility models capture how equity and cryptocurrency option markets actually behave, but they carry a computational curse: the driving noise is fractional, the variance process is non-Markovian, and there is no finite-dimensional pricing partial differential equation (PDE) to solve. The Markovian lift buys a PDE back by approximating the fractional kernel with n exponential factors—at the price of turning a two-dimensional problem into an $(n+1)$ -dimensional one. We take this on with a physics-informed neural network (PINN) that learns the price by minimizing the strong-form PDE residual, and we ask a sharp question: as the lifted dimension grows, does the network’s own accuracy collapse the way a finite-difference grid would? It does not. Measuring the network’s *solve-error* against an independent lifted Monte Carlo at the same n , and separating it cleanly—in price space—from the model-approximation *lift-error*, we find the solve-error grows only sub-linearly, staying under about eleven price basis points out to $n=32$, whereas a naive tensor grid would need 50^{n+1} points and is already infeasible by $n=8$. The crossover between the two error sources gives a practical rule for choosing n , and we show it shifts with maturity. Calibrated to real Bitcoin options, the model returns a genuinely rough Hurst exponent near 0.09 with strong negative skew. Finally we build a parametric PINN that takes the model parameters as inputs, holding the spot variance and maturity fixed; on its own it is too coarse for production calibration, but—and this is the payoff—using it to warm-start a handful of true-model polish steps recovers full from-scratch calibration quality at roughly a ninth of the cost. The takeaway is that where grid methods succumb to dimension, a mesh-free PINN degrades gracefully, and its greatest practical value is as a differentiable warm-start for calibration.

Keywords: physics-informed neural networks; rough volatility; option pricing; Markovian lift; curse of dimensionality; model calibration; scientific machine learning.

1 Introduction

Every derivative-pricing problem is, underneath, a PDE. For the workhorse models of quantitative finance that PDE is low-dimensional and a finite-difference grid dispatches it in milliseconds. Rough volatility breaks this comfortable picture. Driven by the empirical roughness of realized variance and by the steep short-maturity skew that classical models such as Heston (1993) cannot reproduce (Gatheral et al., 2018; Bennedsen et al., 2022; Bayer et al., 2016), and with microstructural foundations in high-frequency trading (El Euch et al., 2018), its variance is driven by a fractional noise with Hurst exponent $H < 1/2$. That single fact—an idea whose lineage runs back to long-memory

volatility (Comte and Renault, 1998)—makes the variance neither Markovian nor a semimartingale, and leaves *no* finite-dimensional PDE to discretize at all (Gatheral et al., 2018; El Euch and Rosenbaum, 2019). The standard escape is the Markovian lift: approximate the fractional kernel by a sum of n exponentials and recover a genuine $(n+1)$ -dimensional PDE (Abi Jaber and El Euch, 2019; Abi Jaber, 2019). But the escape has a cost. It trades an intractable problem for a high-dimensional one, and the accuracy knob n sits directly on top of the PDE dimension. That coupling is exactly what makes the lifted rough-volatility PDE such a clean stress test for mesh-free neural solvers and for the curse of dimensionality—and it is the test we run here.

Physics-informed neural networks are built for this setting. Rather than laying down a grid, a PINN represents the price as a neural network and trains it to satisfy the PDE at randomly sampled collocation points (Raissi et al., 2019). Nothing in the method scales with a grid, so—like the deep Galerkin and deep BSDE solvers before it (Sirignano and Spiliopoulos, 2018; Han et al., 2018)—it sidesteps the exponential blow-up that kills finite differences in high dimensions. For pricing it brings two further advantages we lean on: the physics is known exactly, so no labelled prices are needed; and the trained network is differentiable in its inputs, which we later exploit to calibrate by gradient descent. We run every experiment on Bitcoin (BTC) options, a liquid market with a pronounced negative skew that, as we show, calibrates to a firmly rough Hurst exponent.

We state our contributions exactly as strongly as the evidence allows, and no stronger.

1. **A dimension-scaling result (our headline).** The network’s *solve-error*—how well it actually solves the PDE it is handed—grows only *sub-linearly* as the lifted dimension climbs, staying below roughly eleven price basis points out to $n = 32$, while a naive full-tensor grid explodes as 50^{n+1} and is infeasible by $n = 8$. We are careful about the comparison: 50^{n+1} is the *naive* grid, and structured schemes do better but still degrade with n , so our claim is that finite differences here are *impractical*, not impossible.
2. **An error decomposition and a rule for n .** We split the error into the part the network controls (*solve-error*, versus a lifted Monte Carlo at the same n) and the part only more factors can fix (*lift-error*, versus the true model), report them in price space where they add exactly, and turn their crossover into a concrete rule for choosing n that shifts with maturity.
3. **A real calibration.** Fit to the live Deribit BTC surface, rough Heston returns a rough Hurst exponent with strong negative correlation.
4. **Fast recalibration, honestly.** A parametric PINN plus a warm-start hybrid recovers from-scratch calibration quality at about nine times fewer true-model evaluations—and we are candid that the network on its own does not get there.

We are also clear about what is *not* new. We are not the first to solve the lifted (rough) Heston PDE with a mesh-free network: Papantoleon and Rou (2025) got there first, by a different route. What we add is the combination—a residual PINN, a quantitative scaling-and-decomposition analysis, a calibration to real crypto data, and the recalibration hybrid—which we develop next.

2 Related work

Mesh-free deep pricing on the lifted PDE (the closest work). Papantoleon and Rou (2025) price European options in (rough) diffusion models by recasting the pricing PDE as an energy-minimization problem and approximating it in a time-stepping “deep gradient flow,” demonstrated on the lifted Heston model and careful about the large-moneyness asymptotics and known

price bounds. This is the work nearest ours in both target (the lifted rough-vol PDE) and spirit (mesh-free deep pricing), and we cite it precisely so as not to overreach. It differs in method—theirs is a time-stepping variational scheme, ours a PINN that minimizes the strong-form residual with a hard-wired initial condition—and in scope: our dimension-scaling law, solve/lift decomposition, market calibration, and recalibration hybrid are not its concerns.

Other neural solvers for rough volatility. Jacquier and Zurič (2023) attack the same model class from a very different angle, casting the path-dependent pricing equations as backward stochastic differential equations and solving them with a reservoir-type *random* network, which collapses training to a least-squares regression with convergence guarantees. We share the target but not the mechanism, and neither dimension scaling nor calibration speed is studied there.

Neural calibration surrogates (the foil). The dominant recipe for fast (rough-)volatility calibration is the supervised surrogate of Bayer et al. (2019) and Horvath et al. (2021): train a network offline on a large grid of precomputed prices or implied volatilities, then use it as a fast pricing map inside a calibration loop. It is purely data-driven and carries no PDE consistency—the network never sees the governing equation. Ours is *physics-informed*: it learns from the residual, not from precomputed labels. That distinction is what makes our hybrid work—the physics-trained network supplies a warm start that a few true-model evaluations then sharpen (Section 4.4).

Foundations. We build on PINNs (Raissi et al., 2019) and, more broadly, mesh-free deep solvers for high-dimensional PDEs (Sirignano and Spiliopoulos, 2018; Han et al., 2018); on rough volatility (Gatheral et al., 2018; El Euch and Rosenbaum, 2019) and its multifactor Markovian lift (Abi Jaber and El Euch, 2019; Abi Jaber, 2019); and on classical transform pricing (Heston, 1993; Carr and Madan, 1999) for our independent ground truth. Two training ingredients are worth flagging up front, because PINNs are notorious for optimization pathologies (Krishnapriyan et al., 2021; Wang et al., 2022): self-adaptive loss weighting by homoscedastic uncertainty (Kendall et al., 2018; McClenny and Braga-Neto, 2023), and causal (curriculum) training in time (Wang et al., 2024).

3 Methods

3.1 Rough Heston and the Markovian lift

Under rough Heston the variance is driven by the fractional kernel $K(t) = t^{\alpha-1}/\Gamma(\alpha)$ with $\alpha = H + \frac{1}{2}$ and $H < \frac{1}{2}$, which is non-Markovian (Gatheral et al., 2018; El Euch and Rosenbaum, 2019). The lift (Abi Jaber and El Euch, 2019; Abi Jaber, 2019) replaces it with $K(t) \approx \sum_{i=1}^n c_i e^{-x_i t}$, introducing n Ornstein–Uhlenbeck factors U^i with $V = V_0 + \sum_i c_i U^i$. The discounted price $P(t, x, u)$, with $x = \log S$, then solves the $(n+1)$ -dimensional backward Kolmogorov PDE

$$\begin{aligned} 0 = & P_t + \left(r - \frac{1}{2}V\right)P_x + \frac{1}{2}V P_{xx} + \sum_i \left[-x_i u_i + \lambda(\theta - V)\right]P_{u_i} \\ & + \frac{1}{2}\nu^2 V \sum_{i,j} P_{u_i u_j} + \rho\nu V \sum_i P_{xu_i} - rP, \end{aligned} \quad (1)$$

with terminal condition $P(T, x, u) = \text{payoff}(e^x)$. All factors share one Brownian motion, which is why the full double sum and the cross term appear. The knob n therefore *is* the PDE dimension. We build (c_i, x_i) by the geometric-quadrature rule of Abi Jaber and El Euch (2019); its relative L^2 kernel error falls monotonically from 25.6% at $n = 5$ to 0.39% at $n = 100$, so raising n genuinely buys model fidelity.

3.2 Two independent ground truths

We hold the network to two references that we trust independently. The first is a semi-analytic *Fourier* pricer: it solves the fractional Riccati equation for the rough Heston characteristic function (El Euch and Rosenbaum, 2019) and inverts it by Fourier integration (Carr and Madan, 1999). It never uses the lift, so it is the ground truth of the *true model* ($n \rightarrow \infty$); in the flat-volatility limit it reproduces Black–Scholes to 3.7×10^{-7} . The second is a *lifted Monte Carlo* simulator that integrates the n -factor lifted SDE with an exponential-integrator Euler scheme handling the stiff $-x_i U^i$ term exactly—without which the scheme diverges, since the mean reversion spans some ten orders of magnitude for small H . It matches Black–Scholes within Monte Carlo error. Because the two approximate different objects (an ODE versus the full dynamics), their agreement is a real cross-check. The subtle point we exploit throughout: *at finite n the lifted MC is the ground truth of the PDE the network solves*, while Fourier is the ground truth of the model—and conflating the two is a trap we return to in Section 3.4.

3.3 The physics-informed network

We solve (1) with a PINN (Raissi et al., 2019). The network is a tanh multilayer perceptron of (τ, x, u) with $\tau = T - t$, so the terminal condition becomes an initial condition at $\tau = 0$. Rather than penalize that condition softly, we bake it in through the ansatz

$$P(\tau, x, u) = P_{\text{base}}(\tau, x) + \tau N_{\theta}(\tau, x, u), \quad (2)$$

whose τ prefactor forces the learned correction N_{θ} to vanish at $\tau = 0$ automatically. We anchor the baseline P_{base} not at the spot variance V_0 but at the model’s expected *integrated* variance $w(\tau) = \int_0^{\tau} \mathbb{E}[V_s] ds$, which already captures the mean reversion of V from V_0 toward θ ; anchoring at V_0 leaves a systematic level bias for the network to clean up, and we saw exactly that bias before we fixed the anchor.

We add asymptotic boundary losses— $P \rightarrow 0$ as $S \rightarrow 0$ and $P \rightarrow S - Ke^{-r\tau}$ as $S \rightarrow \infty$ —resampled each iteration. Because PINNs so readily converge to the wrong solution when loss terms sit on very different scales or when temporal causality is ignored (Krishnapriyan et al., 2021; Wang et al., 2022), we add two stabilizers that proved decisive. First, we balance the residual and the two boundary terms with self-adaptive per-term weights, using the learnable homoscedastic-uncertainty scheme of Kendall et al. (2018) in the self-adaptive spirit of McClenny and Braga-Neto (2023), so we never hand-tune a weight. Second, a τ -curriculum trains short maturities first and grows the horizon, respecting causality in time (Wang et al., 2024). One more ingredient mattered more than we expected: we do not sample the factors u in a fixed symmetric box, but draw each from the mean and spread it actually reaches under the lifted dynamics, measured by a short pre-flight Monte Carlo. Getting this wrong flattens the smile; getting it right restores the skew.

3.4 How we score the network, and one trap we avoid

We train the network at fixed strike $K = 1$ and price as a function of spot, then read off the fixed-spot smile $C(1, K)$ through the call’s degree-one homogeneity,

$$C(1, K) = K C(1/K, 1), \quad (3)$$

which holds because the variance dynamics do not depend on the spot level. Equation (3) is exact, not an approximation, and we apply it consistently in every comparison in this paper. We stress this because a protocol ablation—dropping the rescale and comparing the mismatched options

directly—inflates the apparent error to about 2700 volbp. That number is a property of the broken comparison, not a residual error of our method; our accuracy is always the decomposed quantity we now define. Rather than a single “PINN-versus-Fourier” figure, we split the error per strike, in price space where the signed contributions add exactly,

$$\underbrace{(\text{PINN} - \text{Fourier})}_{\text{total}} = \underbrace{(\text{PINN} - \text{MC})}_{\text{solve-error}} + \underbrace{(\text{MC} - \text{Fourier})}_{\text{lift-error}}. \quad (4)$$

The solve-error is ours to improve through training, architecture, and sampling; the lift-error is a model approximation the network cannot touch and only n can shrink. We report price RMSE in basis points of spot, where (4) is additive, and quote implied-vol equivalents where they aid intuition; the two components carry opposite signs, so they do not add in RMS.

3.5 The parametric network

For instant recalibration we train a single network that also takes the five model parameters $(H, \nu, \rho, \lambda, \theta)$ as inputs, over a box around the calibrated BTC values ($H \in [0.04, 0.20]$, $\nu \in [0.10, 0.60]$, $\rho \in [-0.95, -0.20]$, $\lambda \in [0.5, 4.0]$, $\theta \in [0.08, 0.35]$), holding the spot variance V_0 and the maturity fixed. Since the lift weights depend on H , we precompute them on an H -grid and gather per collocation point while the parameters enter the network as exact continuous inputs; we fix the factor count by the crossover rule of Section 4.1 ($n = 10$). Crucially, the network is differentiable in the model parameters, which is what lets us calibrate by gradient descent.

3.6 Calibration procedures and data

We calibrate three ways and compare them head to head: (a) *Fourier-in-the-loop*, differential evolution with the Fourier pricer inside the objective; (b) *PINN gradient*, Adam on the parameters using the parametric network as a differentiable pricer, with zero Fourier calls; and (c) *hybrid*, the PINN’s single-shot solution as an initial guess, sharpened by a few local Fourier polish steps (Nelder–Mead). The objective throughout is the vega-weighted implied-volatility RMSE against the market smile. The market data are end-of-day BTC option quotes from Deribit, a cryptocurrency derivatives exchange and data provider, retrieved through its public interface; we set $r = 0$ and infer the forward from the quotes, as is standard for crypto, and recompute implied volatilities from mid prices with a robust Black–Scholes inverter.

4 Results

4.1 The network scales where the grid cannot

Our headline experiment asks what happens to the network’s own accuracy as we push the lifted dimension. We train one architecture (width 64, depth 4) at $n = 4, 8, 16, 32$ on the canonical BTC calibration at $T = 0.15$ and measure the solve-error against the lifted MC over three seeds (Table 1, Figure 1). The answer is encouraging: the solve-error creeps up only mildly, from 5.4 ± 1.0 to 10.7 ± 2.0 price bp—about a doubling for an eight-fold jump in dimension. We resist the temptation to call it flat; an early single-seed run made it look flat, and three seeds corrected us. With only three seeds the error bars at adjacent n overlap, so we read the trend across the full range rather than claiming significance between neighbours, and the robust statement is that two-fold growth from $n = 4$ to $n = 32$. Wall-clock to the plateau stays bounded throughout (about 6–10 minutes on a CPU, with per-iteration cost essentially flat at 55–62 ms) because the factor Hessian is vectorized.

Table 1: Dimension scaling (canonical BTC calibration, $T = 0.15$; solve-error is 3-seed mean $\pm$ SD; price RMSE in basis points of spot).

n	solve-error (px bp)	lift-error (px bp)	train time (s)	FD grid $50^{(n+1)}$
4	5.4 ± 1.0	16.4	397	3.1×10^8
8	6.1 ± 1.4	7.9	386	2.0×10^{15}
16	8.8 ± 1.6	3.5	591	7.6×10^{28}
32	10.7 ± 2.0	1.7	457	1.2×10^{56}

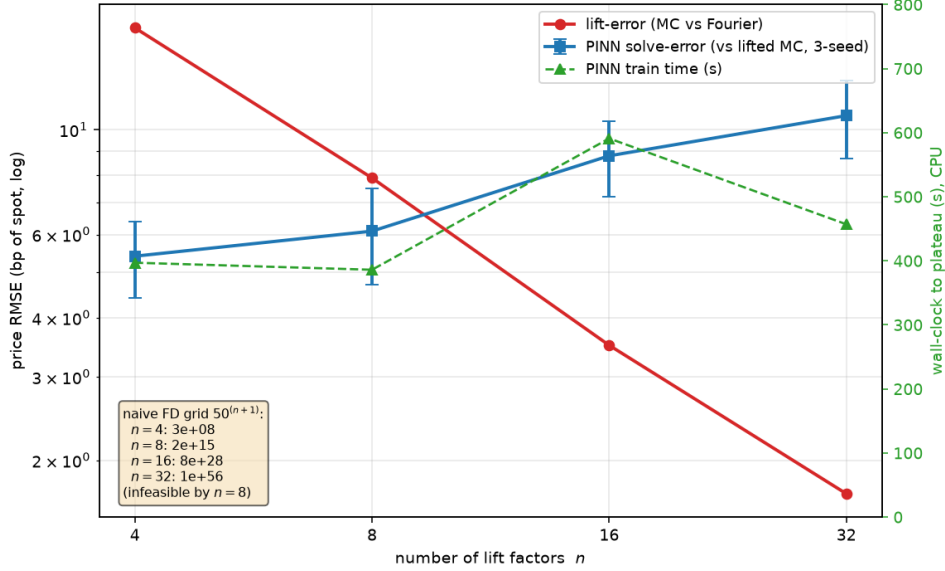


Figure 1: Dimension scaling. The PINN solve-error (blue, 3-seed mean $\pm$ SD) grows sub-linearly and stays under ~ 11 price bp to $n = 32$, while the lift-error (red) falls monotonically and training time (green) stays bounded; the naive finite-difference grid (inset) is infeasible by $n = 8$.

Set this against the alternative. A naive full-tensor finite-difference grid on the same $(n+1)$ -dimensional PDE, at a modest 50 nodes per dimension, needs 50^{n+1} points: 3.1×10^8 at $n = 4$, 2.0×10^{15} at $n = 8$, and 1.2×10^{56} at $n = 32$. Even $n = 8$ —about 16 petabytes in double precision—is out of reach, while the network trains every n on a laptop. We are deliberately fair about this contrast: 50^{n+1} is the *naive* grid, and structured schemes (sparse grids, ADI, low-rank tensor-train) reach further, but their cost and accuracy still degrade with n —sparse-grid node counts grow super-linearly, and tensor-train needs a low-rank structure that the coupled ρ - ν cross terms erode—and none is standard or validated for this PDE. So the honest verdict is that a competitive finite-difference solve at $n = 16$ – 32 is *impractical*, not provably impossible. Even in that conservative form, gentle solve-error growth with bounded cost, against a grid that cannot be stored, is precisely the curse-of-dimensionality advantage that motivates going mesh-free.

4.2 Splitting the error, and a rule for how many factors to use

The decomposition did more than tidy our bookkeeping—it overturned how we read our own numbers. Taken at face value, a single “PINN vs Fourier” implied-vol RMSE of 114–127 vol bp at

$n = 4$ looks like network error. Split by (4), almost all of it is the lift. In the full configuration the $n = 4$ error is 37 vol bp of solve-error on top of 133 of lift (they partly cancel, to a total of 121), and with correlation switched off it is a mere 8 of solve-error under 133 of lift. In other words, the network already solves its PDE to well under 50 vol bp; what had looked like a modelling failure of the network was the finite- n model catching up to reality.

Because the lift-error shrinks monotonically with n (133, 63, 27, 12 vol bp at $n = 4, 8, 16, 32$; equivalently $16.4 \rightarrow 1.7$ price bp) while the solve-error climbs gently, the two curves cross. At $T \approx 0.15$ they meet near $n = 8$ –10 (Figure 1), and that crossing is a genuinely useful design rule: below it, add factors, because model approximation dominates; above it, stop, because you are only paying for network error. The crossover is not fixed—it tracks maturity. Over a T -sweep the solve-error stays small and bounded everywhere (roughly 1–9.5 price bp across $T \in \{0.05, 0.15, 0.5\}$ and $H \in \{0.05, 0.10, 0.15\}$), but the lift-error grows with T —from 8.6 to 31.8 price bp at $n = 4$ as T runs $0.05 \rightarrow 0.5$ —so at long maturities the lift still tops the solve floor at $n = 16$ and the crossover slides to larger n . Longer horizons simply need more factors.

4.3 Bitcoin really is rough

We put the model to real data. Calibrated to the live Deribit BTC surface with a full-budget optimizer (differential evolution, 60 generations plus local polish; all 13 maturities, up to 8 points each, 97 points), rough Heston returns $H = 0.090$, $\rho = -0.792$, $V_0 = 0.103$, $\theta = 0.253$, $\lambda = 2.567$, $\nu = 0.300$, at a vega-weighted implied-vol RMSE of 496 vol bp (Figure 2). The Hurst exponent lands firmly in the rough regime and the correlation is strongly negative—both exactly what one expects of a crypto market with a steep, persistent skew. We do not dress up the fit: at 496 vol bp it is coarse, and the per-maturity smiles in Figure 2(b) show where it strains. That coarseness is a statement about the model’s expressiveness on wide, richly curved crypto smiles, not about our solver, and it is orthogonal to the solve-error we have been tracking.

4.4 The parametric network, and the hybrid that makes it pay off

A single network stretched over a five-parameter box cannot match a network trained at one point, and we quantify the price of coverage. The parametric network varies $(H, \nu, \rho, \lambda, \theta)$ while holding V_0 and T fixed; at the box centre its solve-error is 11.0 ± 4.7 price bp against 5.9 ± 1.2 for a single-point network at the same $n = 10$ —roughly a two-fold penalty. And the penalty is uneven: it hovers near 9–11 price bp at typical points but climbs to 47.7 in the rough, low- H corner and 29.8 at high vol-of-vol, where the PDE is stiffest. We report this plainly; a larger training budget would narrow the corner gap.

This is where a less careful paper would oversell. Calibrating the BTC $T \approx 0.15$ slice three ways (Table 2, Figure 3) tells the honest story. Fourier-in-the-loop reaches 23.7 vol bp, but pays 2018 Fourier evaluations and about 168 s. The parametric PINN on its own is almost free—about 2 s, zero Fourier calls—but its fit is poor, 222 ± 20 vol bp under the true model, because it calibrates to its own imperfect prices and, tellingly, because this single-slice optimum falls *outside* the training box (it wants $\nu \approx 0.87$, $\theta \approx 0.39$). So we do not sell the standalone network. Instead we use it as a launch point: seed the local Fourier polish with the PINN’s guess, and it reaches 29.4 ± 7.1 vol bp—on par with from-scratch on a 34–52% smile—in only 237 ± 46 Fourier evaluations and about 23 s. That is roughly nine times fewer true-model calls and seven times faster, for the same practical fit. The parametric network earns its keep not as a pricer but as a differentiable warm start.

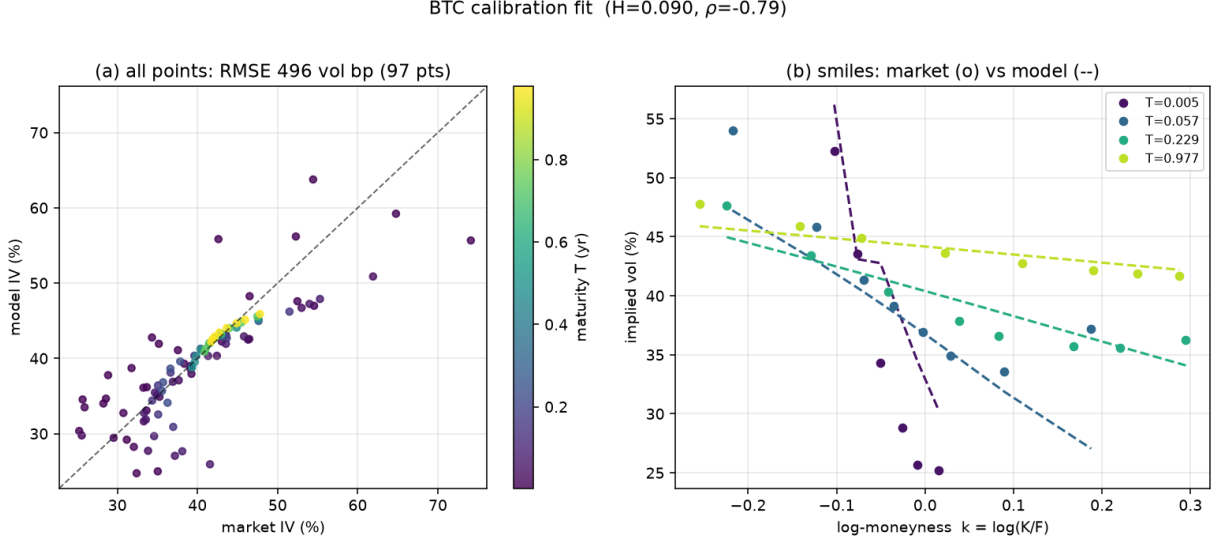


Figure 2: Rough Heston calibrated to the Deribit BTC surface (97 points, 13 maturities) at the canonical fit ($H = 0.090$, $\rho = -0.79$; RMSE 496 vol bp). (a) market versus model implied volatility at all points (dashed line: identity; colour = maturity). (b) per-maturity smiles for four representative maturities: market (markers) versus model (dashed). The fit is deliberately coarse: the residual spread reflects a known limitation of the lifted rough-Heston form on wide, richly curved crypto smiles—a model-expressiveness issue, not a solver artifact—and is orthogonal to the solve-error studied elsewhere.

Table 2: Calibration of the BTC $T \approx 0.15$ slice, three ways (3 PINN seeds; all judged by the true Fourier model over the same feasible region).

method	fit RMSE (vol bp)	wall-clock (s)	Fourier evals
Fourier-loop from scratch	23.7	168	2018
PINN-only (single-shot)	222 ± 20	~ 2	0
Hybrid (PINN + Fourier polish)	29.4 ± 7.1	23	237 ± 46

5 Discussion

What the scaling result does and does not say. The finding we care about most is that the network’s own error in solving the lifted PDE degrades only gently as the dimension grows, while grid methods become impossible. We have taken pains to state it in its honest, weak form. The solve-error is not constant in n —a claim an early single-seed run seemed to support and three-seed statistics refuted—it grows sub-linearly, roughly doubling from $n = 4$ to $n = 32$. And 50^{n+1} is the naive grid; sparse-grid, ADI, and tensor-train methods reach higher dimensions, so the defensible claim is that a competitive finite-difference solve for this PDE at $n = 16$ – 32 is impractical, not impossible. Even so, the contrast—sub-linear error and bounded wall-clock against a grid that cannot be stored—is a concrete instance of the advantage that motivates mesh-free neural solvers, and it is the cleanest such instance we know of on an economically real PDE.

A lesson beyond rough volatility. The most transferable idea here is methodological. When a surrogate replaces an exact solver, it is dangerously easy to validate it against the wrong reference.

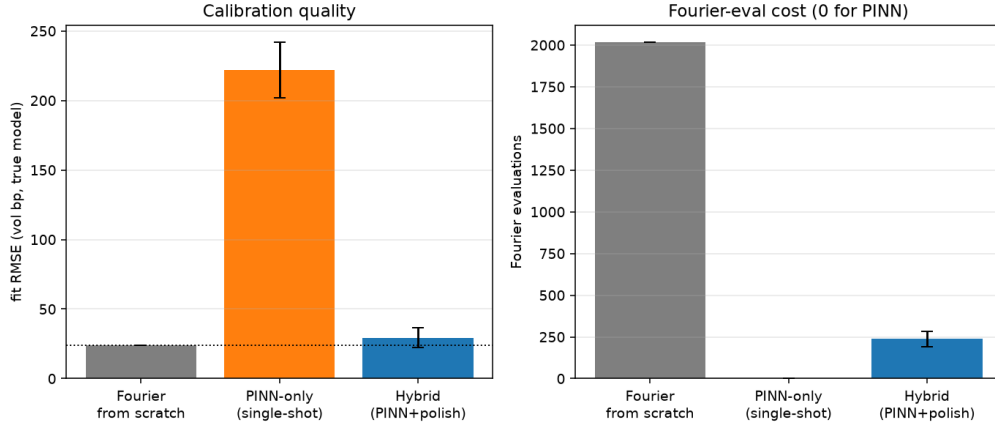


Figure 3: Hybrid calibration. Fit quality (left) and Fourier-evaluation cost (right) for the three methods on the BTC slice: the hybrid matches from-scratch quality at $\sim 9\times$ fewer Fourier evaluations, while the PINN alone is fast but inaccurate.

Our network solves the finite- n lifted PDE, whose exact solution is the lifted Monte Carlo, not the $n \rightarrow \infty$ model; comparing straight to the true model conflates the network’s solve-error with the model’s lift-error. We found the two have opposite sign and that the historical “PINN-versus-truth” gap was lift-dominated, not the network’s doing. Validate against a same-fidelity reference, report the two errors separately, and let their crossover set the approximation order: none of this is special to rough volatility, and we would urge it on anyone placing a learned solver on top of an approximated operator.

The calibration story, straight. We are deliberately blunt about a negative result and its fix. One network spread over five parameters is markedly less accurate than a point-trained one, worst in the rough, high-vol-of-vol corners, and calibrating with it alone is fast but unreliable—it fits its own imperfect prices, and the single-slice optimum can sit outside the training box. Yet the same network is a fine *warm start*: seeding a short true-model polish with its guess recovers from-scratch quality at about an order of magnitude fewer true-model evaluations. The value proposition is amortized—train once, then recalibrate cheaply—with the exact pricer kept in the loop for a brief, decisive finish.

Limitations. Our evidence rests on a single market snapshot (one Deribit BTC surface), and the parametric and calibration studies use a single maturity slice; timings are single-machine CPU figures at moderate training budgets. The parametric network fixes V_0 and the maturity, its surface-level fit to the wide crypto smile is coarse (496 vol bp), and its solve-error carries a mild directional bias—a slight over-pricing of at-the-money and call strikes that grows with n . We study $n \leq 32$; the sub-linear trend need not continue arbitrarily far.

Where we take it next. Several extensions are immediate: add the maturity as a network input to price whole surfaces rather than single slices; scale width, depth, and GPU training to shrink the parametric corner penalty and probe larger n ; carry the approach to other rough models (rough Bergomi) and to American and path-dependent payoffs; and map the warm-start polish across market regimes. We would also like to pin down the high- n solve-bias and whether loss

reweighting or a residual correction removes it, and to benchmark the mesh-free solver directly against structured finite differences at the moderate n where the latter still stand a chance.

References

- E. Abi Jaber. Lifting the Heston model. *Quantitative Finance*, 19(12):1995–2013, 2019. doi: 10.1080/14697688.2019.1615113.
- E. Abi Jaber and O. El Euch. Multifactor approximation of rough volatility models. *SIAM Journal on Financial Mathematics*, 10(2):309–349, 2019. doi: 10.1137/18M1170236.
- C. Bayer, P. Friz, and J. Gatheral. Pricing under rough volatility. *Quantitative Finance*, 16(6): 887–904, 2016. doi: 10.1080/14697688.2015.1099717.
- C. Bayer, B. Horvath, A. Muguruza, B. Stemper, and M. Tomas. On deep calibration of (rough) stochastic volatility models, 2019. Preprint, arXiv:1908.08806.
- M. Bennedsen, A. Lunde, and M. S. Pakkanen. Decoupling the short- and long-term behavior of stochastic volatility. *Journal of Financial Econometrics*, 20(5):961–1006, 2022. doi: 10.1093/jjfinec/nbaa049.
- P. Carr and D. B. Madan. Option valuation using the fast Fourier transform. *Journal of Computational Finance*, 2(4):61–73, 1999. doi: 10.21314/JCF.1999.043.
- F. Comte and E. Renault. Long memory in continuous-time stochastic volatility models. *Mathematical Finance*, 8(4):291–323, 1998. doi: 10.1111/1467-9965.00057.
- O. El Euch and M. Rosenbaum. The characteristic function of rough Heston models. *Mathematical Finance*, 29(1):3–38, 2019. doi: 10.1111/mafi.12173.
- O. El Euch, M. Fukasawa, and M. Rosenbaum. The microstructural foundations of leverage effect and rough volatility. *Finance and Stochastics*, 22(2):241–280, 2018. doi: 10.1007/s00780-018-0360-z.
- J. Gatheral, T. Jaisson, and M. Rosenbaum. Volatility is rough. *Quantitative Finance*, 18(6): 933–949, 2018. doi: 10.1080/14697688.2017.1393551.
- J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.
- S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993. doi: 10.1093/rfs/6.2.327.
- B. Horvath, A. Muguruza, and M. Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. *Quantitative Finance*, 21(1):11–27, 2021. doi: 10.1080/14697688.2020.1817974.
- A. Jacquier and Ž. Zurič. Random neural networks for rough volatility, 2023. Preprint, arXiv:2305.01035.

- A. Kendall, Y. Gal, and R. Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7482–7491, 2018. doi: 10.1109/CVPR.2018.00781.
- A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 34:26548–26560, 2021.
- L. D. McClenny and U. M. Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023. doi: 10.1016/j.jcp.2022.111722.
- A. Papapantoleon and J. Rou. A time-stepping deep gradient flow method for option pricing in (rough) diffusion models. *Quantitative Finance*, 25(12):2009–2020, 2025. doi: 10.1080/14697688.2025.2572318.
- M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- J. Sirignano and K. Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018. doi: 10.1016/j.jcp.2018.08.029.
- S. Wang, X. Yu, and P. Perdikaris. When and why PINNs fail to train: a neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi: 10.1016/j.jcp.2021.110768.
- S. Wang, S. Sankaran, and P. Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024. doi: 10.1016/j.cma.2024.116813.